

CS412 Fuzzing Lab

Helen El Khoury
360551

Mathilde Peruzzo
346576

Leila Sidjanski
330810

Alexandre Spiess
342757

Introduction

We fuzzed libpng 1.2.56 with AFL++, running two campaigns: a white-box campaign using compile-time instrumentation and AddressSanitizer, and a black-box campaign using AFL++ QEMU mode. Both used the same harness targeting the full PNG decoding pipeline, a seed corpus from the official libpng test suite, and a PNG chunk-type dictionary. Neither campaign found real crashes; we validated our setup by injecting a synthetic off-by-one heap write that AFL++ and ASan detected correctly. Our artifacts are available at: <https://github.com/helkhour/cs412-fuzzing-lab/tree/main>.

1 Question 1 – Harness Design

We chose the libpng decoding pipeline as our entry point, centered around `png_read_info`, `png_read_image` and `png_read_end`. We chose the decoder over the encoder because real applications receive PNG files from untrusted external sources and pass them directly into the decoder. Since this is the path an application follows when opening a PNG file, it is also the most realistic attack vector from an attacker's perspective. We also chose the high-level API over the low-level one because it covers a large portion of the attack surface in a single execution: header parsing, decompression, pixel decoding and post-image metadata.

We considered two alternatives. The low-level chunk interface was rejected because even though it gives more control over which chunks are read, both the low-level and high-level interfaces ultimately call the exact same internal chunk handler functions inside libpng. The actual parsing code that runs for each chunk is identical either way, so there is no coverage advantage, only added harness complexity. The encoder pipeline was rejected because it does not read from a file at all. It takes pixel data that is already structured in memory by the harness itself. Since AFL++ works by mutating input files, those mutations never actually reach the encoder, making it a poor fuzzing target. You could work around this by writing a

harness that maps file bytes to encoder parameters, but this would require significant engineering effort for a target that is less security relevant than the decoder, since in practice attackers send malicious files to be decoded, not pixel buffers to be encoded.

The data flow through the harness is straightforward. AFL++ mutates a file and passes its path to the harness via `@@`. The harness opens it with `fopen` and connects it to libpng using `png_init_io`, so every subsequent library call reads directly from the mutated file. `png_read_info` is the first function to consume bytes from it, parsing the PNG signature and all pre-IDAT chunks. `png_read_image` then reads and decompresses the pixel data. Finally, `png_read_end(png, info)` processes all remaining post-IDAT chunks, which is where `png_set_text_2` gets called and where CVE-2016-10087 can be reached.

The harness includes several guards to make sure AFL++ only records real bugs and does not get stuck. The file open and struct creation checks (lines 20, 29, 37) return 0 on failure so that system-level errors are never mistaken for crashes. The `setjmp` handler (line 48) is the most important one: without it, every time libpng encounters a malformed input it calls `longjmp` internally, which AFL++ would interpret as a crash. This would fill the findings directory with thousands of false positives that have nothing to do with real bugs. The dimension cap of 4096 pixels (line 77) protects against mutated headers that claim a huge image size, which would cause the harness to run out of memory and hang. The per-row allocation check (line 120) makes sure that if something goes wrong during memory allocation, everything is cleaned up properly so the fuzzer can keep running.

2 Question 2 – Instrumentation and Sanitizers

For the white-box fuzzing campaign, we applied one patch and three compiler flags to build libpng 1.2.56 with AFL++.

We first applied the AFL++ CRC patch: `patch -forward -p0 < patches/libpng-nocrc.patch`. This patch removes the normal CRC check in libpng. Without the patch, most mu-

tations made by AFL++ would cause a checksum mismatch, so libpng would reject many mutated PNGs very early, before reaching more interesting parsing code.

Then, we compiled libpng with the following compiler flags: `-fsanitize=address`, `-g`, and `-O1`.

The flag `-fsanitize=address` enables AddressSanitizer. It adds runtime checks for memory safety bugs such as buffer overflows and use-after-free. Without ASan, some memory bugs might not crash and may go unnoticed.

The flag `-g` adds debug information. This helps ASan map memory addresses back to source file names, function names, and line numbers. Without it, ASan may only show raw hexadecimal addresses, making crashes harder to understand.

The flag `-O1` enables basic optimizations. This improves performance and allows AFL++ to test more inputs per second. Without it, the target would likely run slower during fuzzing.

3 Question 3 – Seed Corpus and Dictionary

For the seeds, we need a diverse set of png images with various encoding combinations. We used Willem van Schaik's PNG Suite [5], which is the official libpng set of test images. It contains different combinations of PNG features: RGB / greyscale / palette, interlaced / non-interlaced, with / without alpha, with various depth and various types of ancillary chunks. We minimized it using AFL++ with the command `afl-cmin -i orig_seeds/ -o seeds/ -./png_fuzz @@`. Note that this command is not reproducible in the environment as the `orig_seeds` directory is absent from our repository. After that we obtained a set of 60 images.

A PNG image must follow a specific structure (its format), which consists of chunks, each containing 4 bytes to specify its type. This format can be interpreted as a grammar, where the chunk types are symbols. The role of a dictionary is to provide those symbols to the fuzzer to help it generate random images conforming to the grammar.

The dictionary is as follows:

- **The PNG signature:** This is the byte sequence that must be present at the start of each PNG image. `header_png="\x89PNG\x0d\x0a\x1a\x0a"`.
- **The critical chunks types:** Those are the chunks that must be present in every PNG image (except PLTE which is optional for color types 2 and 6).

```
section_IHDR="IHDR"      section_IEND="IEND"
section_IDAT="IDAT"      section_PLTE="PLTE"
```

- **The ancillary chunk types:** Those are optional chunks that serve different purposes. We included the chunks types presented in the lab description.

```
section_gAMA="gAMA"      section_bKGD="bKGD"
section_cHRN="cHRN"      section_tEXt="tEXt"
section_iTXt="iTXt"      section_tIME="tIME"
section_pHYs="pHYs"      section_tRNS="tRNS"
section_sRGB="sRGB"      section_zTXt="zTXt"
```

In our AFL++ status screen (Figure 1), we can see that the dictionary contributed 0 new coverage, despite trying 19.8k times. The havoc/splice strategy had better results, having found 338 new paths after trying 538k times.

4 Question 4 – Campaign Analysis

We ran our instrumented campaign for approximately 40 minutes. The AFL++ status screen (Figure 1) and the edges graph (Figure 2) are included in the appendix.

The four key metrics from the status screen are as follows. Stability was 100%, meaning the harness behaves completely deterministically and every input produces the same coverage every time it is run. The corpus count reached 624 inputs, meaning AFL++ found 624 distinct inputs that each trigger at least one new code path. Cycles done was 5, meaning AFL++ went through its entire corpus 5 times. Map density was 9.92%/28.52%, where the first number reflects the density at the time of the screenshot and the second represents the peak density reached during the campaign. This means that at its peak, the fuzzer explored approximately 28.52% of all instrumented edges in the binary.

The edges graph shows a steep initial climb from approximately 775 edges to around 810 edges within the first 100 seconds, then continuing to rise to approximately 865 edges by 500 seconds. After that the curve flattens significantly, with small occasional steps bringing the total to around 888 edges by the end of the campaign. This shape is typical of a coverage-guided fuzzer: it rapidly discovers easy-to-reach code paths from the initial seeds, then slows down as the remaining undiscovered paths require more complex mutations to reach.

The campaign has not fully reached saturation since the curve still shows occasional small steps rather than being completely flat. However, the rate of new edge discovery dropped sharply after the first 10 minutes, suggesting that most code paths reachable from our seed corpus and dictionary have already been found. The havoc strategy was the dominant contributor, finding 338 new paths out of 538k attempts, while the dictionary contributed no new coverage. Running the campaign for several more hours would likely yield only marginal additional coverage.

5 Question 5 – Crash Triage

During our instrumented campaign, AFL++ did not find any real crashes. To prove that our setup works correctly, we injected a synthetic off-by-one heap write bug into

`png_read_info()` in `pngread.c` (see Listing 1 in the appendix). The bug is triggered when AFL++ mutates the last byte of the IHDR chunk name from R to S. Since valid PNG files always contain IHDR and not IHDS, the initial seeds do not crash immediately and AFL++ has to discover the mutation itself. The injected bug allocates a small heap buffer named `small` with only 4 bytes. The valid indices are `small[0]` to `small[3]`. Writing to `small[4]` writes one byte past the end of the heap buffer, creating an off-by-one heap overflow. The synthetic bug was stored as `patches/synthetic-bug.patch` and can be applied with `make patch-bug`, `make build-libpng`, and `make build-harness`. These commands apply the synthetic bug patch and rebuild libpng and the harness with AFL++ instrumentation and ASan. The normal CRC-removal patch had already been applied as part of the white-box build.

After rebuilding, we re-ran AFL++ and as shown in Figure 3, the crash was found after 27 seconds with `saved_crashes = 1`. This confirms that our fuzzer can reach code inside libpng, generate meaningful mutations, and detect memory safety violations. We then reproduced the crash using the command in Listing 2 in the appendix. As shown in Figure 4, ASan confirms the bug as a heap buffer overflow, reporting a write of size 1 immediately after a 4-byte heap region. The stack trace confirms the crash happens inside `png_read_info()`, called directly from our harness.

6 Question 6 – Attack Surface Analysis

Firefox is a well-known web browser that uses libpng to handle PNG images. While our harness only reads images all at once, Firefox is able to read them progressively as data becomes available using the dedicated libpng API functions. In its `image/decoders/nsPNGDecoder.cpp` [1] and `media/libpng/pngpread.c` [2] files, it calls `png_process_data`, which processes each row of the image as it receives them using callbacks, and `png_set_progressive_read_fn` to set up those callbacks. Like for our harness, this path is important for security because it is used to read user-provided images. An attacker could exploit an out-of-bounds read to access sensitive information by providing an input image able to trigger such a vulnerability in this reading pipeline. Multiple recent libpng CVEs involve this type of bug (for example CVE-2026-22695, though not specifically in the progressive decoding path), which shows that out-of-bound reads are still relevant for security in this library.

ImageMagick [3] is a commonly used library for manipulating images that uses libpng for handling PNGs. It exercises the encoding/writing path which is not exercised by our harness, using functions `png_write_info`, `png_write_row` and `png_write_end` in its `coders/png.c` file [4]. Multiple applications use ImageMagick on user-provided input, and the encoding path is frequently used to write an image af-

ter a transformation. This matters for security as an attacker could choose some specific input image to transform that is able to trigger a bug during the rewriting phase. For example, an attacker could pick an image leading to a crash. Server-side applications using ImageMagick could be vulnerable to denial-of-service attacks if attacker-controlled images trigger crashes during PNG encoding. A recent example is CVE-2026-22801, an integer truncation in libpng’s write API that causes a heap buffer over-read when processing images with unusual row strides.

7 Question 7 – Binary-Only Fuzzing with QEMU Mode

We ran the same libpng harness in two configurations. The white-box campaign used AFL++ compile-time instrumentation and ASan. The binary-only campaign used a second build of the same harness and library compiled with plain `gcc`, without AFL++ instrumentation and without sanitizers, then fuzzed with AFL++ QEMU mode using `afl-fuzz -Q`. This follows the binary-only setup required by the lab, where the target is treated as a closed-source binary.

Both campaigns were run for approximately the same wall-clock time. Whenever possible, the metrics below are taken directly from the AFL++ status screenshots. When the UI only showed rounded values or was temporarily frozen, we used the corresponding internal `fuzzer_stats` logs. The full metrics are summarized in Table 1 in the appendix.

The QEMU status screen is shown in Figure 5, and the white-box status screen is shown in Figure 1. The corresponding `afl-plot` edge graphs are shown in Figure 6 and Figure 2.

In the screenshots, the white-box run lasted 39 minutes and 57 seconds, while the QEMU run lasted 39 minutes and 58 seconds. The white-box status screen reports an instantaneous speed of 535.1 `exec/s` and 898k total executions. The QEMU status screen reports 1.06M total executions, but the speed field shows 0.00 `exec/s` because the AFL++ UI was temporarily idle at the moment of the screenshot. For this reason, we use the average QEMU speed from the internal `fuzzer_stats` logs, which was 440.74 `exec/s`.

The QEMU campaign executed more test cases over the same time window, with approximately 1.06M executions compared to 898k for the white-box run. This is counter to the usual expectation that QEMU mode is slower than compile-time instrumentation. However, this difference can be explained by the build configuration. The white-box binary was compiled with ASan, while the QEMU binary was compiled without sanitizers, as required for the binary-only setup. ASan adds runtime memory-checking overhead, and in this experiment that overhead outweighed the dynamic-translation overhead introduced by QEMU.

QEMU also ended with a larger corpus, with 703 saved

inputs compared to 624 for the white-box campaign. This suggests that, in this particular setup, the QEMU run kept slightly more inputs that exercised new coverage. However, this does not mean that QEMU necessarily explored more source-level behavior in a direct semantic sense, because the two modes do not count edges in the same way.

The difference in map density illustrates this point. The white-box run reached 888 edges out of 3114, corresponding to 28.52% peak map density. The QEMU run reported 2375 edges out of 65536, corresponding to 3.62% peak map density. The QEMU denominator is much larger because AFL++’s QEMU mode uses a different coverage-map representation. Therefore, the absolute edge counts and map densities should be interpreted as mode-specific coverage indicators, not as a one-to-one comparison of the same program edges.

Both campaigns were stable, with 100.00% stability in each case. In our run, QEMU produced more executions and a larger corpus mainly because the binary was unsanitized, while the white-box reference build included ASan.

8 Question 8 – Instrumentation Depth and Performance

For the instrumentation-depth measurement, we used AFL++ compile-time instrumentation. Since AFL++ needs an executable target, the “library alone” measurement was performed using libpng’s own `pngtest` driver linked against the instrumented libpng build. The final harness was measured using our custom PNG fuzzing harness.

The AFL++ screenshots report the peak map density directly. The total number of instrumented edges is not shown explicitly in the UI, so it was taken from the internal AFL++ logs. The reached-edge counts below are therefore computed from the logged total edge count and the peak density visible in the screenshots. The results are summarized in Table 2 in the appendix.

The library-driver build reports more instrumented edges because `pngtest` pulls in and exercises a broader libpng test path, including extra validation, read/write, and checking logic. The final harness mainly drives the PNG read/decode path needed for fuzzing, so the executable contains a smaller effective set of instrumented control-flow edges. The AFL++ screenshots supporting this comparison are provided in the appendix: Figure 7 shows the `pngtest` run, while Figure 8 shows the final harness run.

For the main instrumented fuzzing campaign, AFL++ reported 888 / 3114 edges reached, corresponding to 28.52% peak bitmap coverage. This is expected: not every instrumented edge is reachable from the harness and seed corpus.

For performance, we measured the same harness under three configurations. The ASan fork-mode and ASan persistent-mode values are taken directly from the AFL++ status screenshots.

For the no-sanitizer fork-mode run, no valid screenshot was available, so the values are taken from the internal `fuzzer_stats` logs. The results are summarized in Table 3 in the appendix.

The no-sanitizer fork-mode binary is much faster than ASan fork mode because ASan adds memory-access checks, redzones, allocator instrumentation, and crash-detection overhead.

Persistent mode is fastest despite still using ASan because `__AFL_LOOP` lets AFL++ reuse the same process for many testcases, avoiding most per-input forking server startup and process initialization cost. The persistent run reported 99.72% stability in the screenshot, slightly below 100%, but still usable for this speed comparison. The corresponding AFL++ status screen is shown in Figure 10.

All result files can be found in our GitHub repository under:

- `q8-edges-libpng/default/fuzzer_stats`
- `q8-edges-harness/default/fuzzer_stats`
- `speed-nosan-fork/default/fuzzer_stats`
- `speed-asan-fork/default/fuzzer_stats`
- `speed-asan-persistent/default/fuzzer_stats`

Figures, Listings and Tables

```
american fuzzy lop ++4.41a {default} (/work/png_fuzz) [explore]
├─ process timing ─────────── overall results ───────────
│   run time : 0 days, 0 hrs, 39 min, 57 sec      cycles done : 5
│   last new find : 0 days, 0 hrs, 0 min, 4 sec    corpus count : 624
│   last saved crash : none seen yet               saved crashes : 0
│   last saved hang : none seen yet                saved hangs : 0
├─ cycle progress ─────────── map coverage ───────────
│   now processing : 425*1 (68.1%)                  map density : 9.92% / 28.52%
│   runs timed out : 0 (0.00%)                     count coverage : 3.40 bits/tuple
├─ stage progress ─────────── findings in depth ───────────
│   now trying : havoc                             favored items : 129 (20.67%)
│   stage execs : 94/100 (94.00%)                 new edges on : 167 (26.76%)
│   total execs : 898k                            total crashes : 0 (0 saved)
│   exec speed : 535.1/sec                         total tmouts : 19 (0 saved)
├─ fuzzing strategy yields ─────────── item geometry ───────────
│   bit flips : 3/10.6k, 2/10.6k, 2/10.6k          levels : 6
│   byte flips : 0/1321, 0/1317, 0/1309            pending : 21
│   arithmetics : 7/92.3k, 0/182k, 0/181k           pend fav : 0
│   known ints : 0/11.8k, 0/49.8k, 0/73.1k         own finds : 564
│   dictionary : 0/19.8k, 0/19.9k, 0/0, 0/0        imported : 0
│   havoc/splice : 338/538k, 0/0                   stability : 100.00%
│   py/custom/rq : unused, unused, unused
│   trim/eff : 21.33%/252k, 99.02%
└─ strategy: explore ───────── state: in progress ─────────
```

Figure 1: AFL++ status screen at the end of the white-box campaign (40 minutes, libpng 1.2.56).

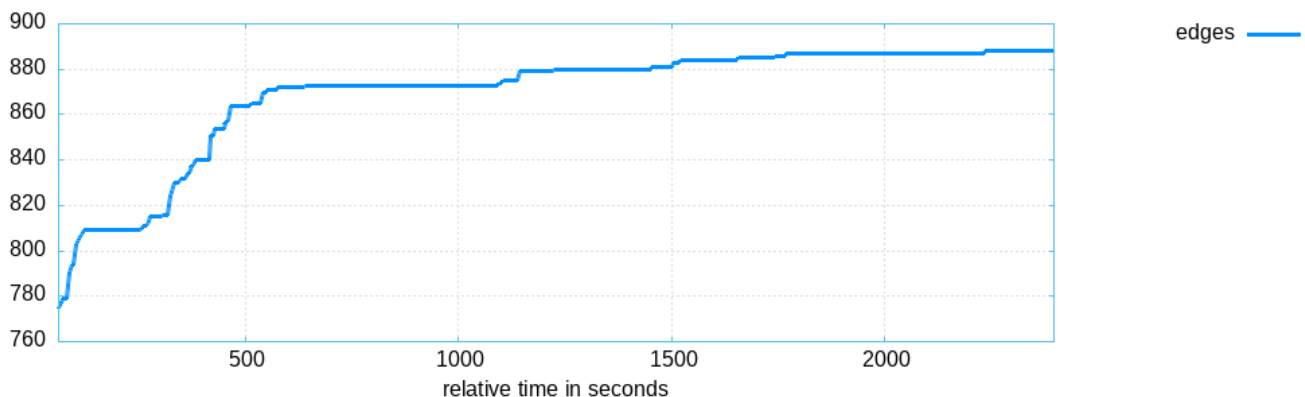


Figure 2: Edges discovered over time during the white-box campaign, generated with `afl-plot`.

Listing 1: Synthetic off-by-one bug injected in png_read_info()

```
if (chunk_name[0] == 'I' &&
    chunk_name[1] == 'H' &&
    chunk_name[2] == 'D' &&
    chunk_name[3] == 'S')
{
    png_bytep small = (png_bytep)png_malloc(png_ptr, 4);
    small[4] = 0x41; /* off-by-one write */
    png_free(png_ptr, small);
}
```

Listing 2: Command to reproduce crash with ASan

```
docker run --rm -v "$(pwd)":/work cs412-libpng-fuzz bash -lc '
CRASH=$(ls_/work/findings/default/crashes/id:*_|_head_-n1)
ASAN_OPTIONS=symbolize=1:detect_leaks=0:abort_on_error=1_\
/work/png_fuzz_"$CRASH"
'
```

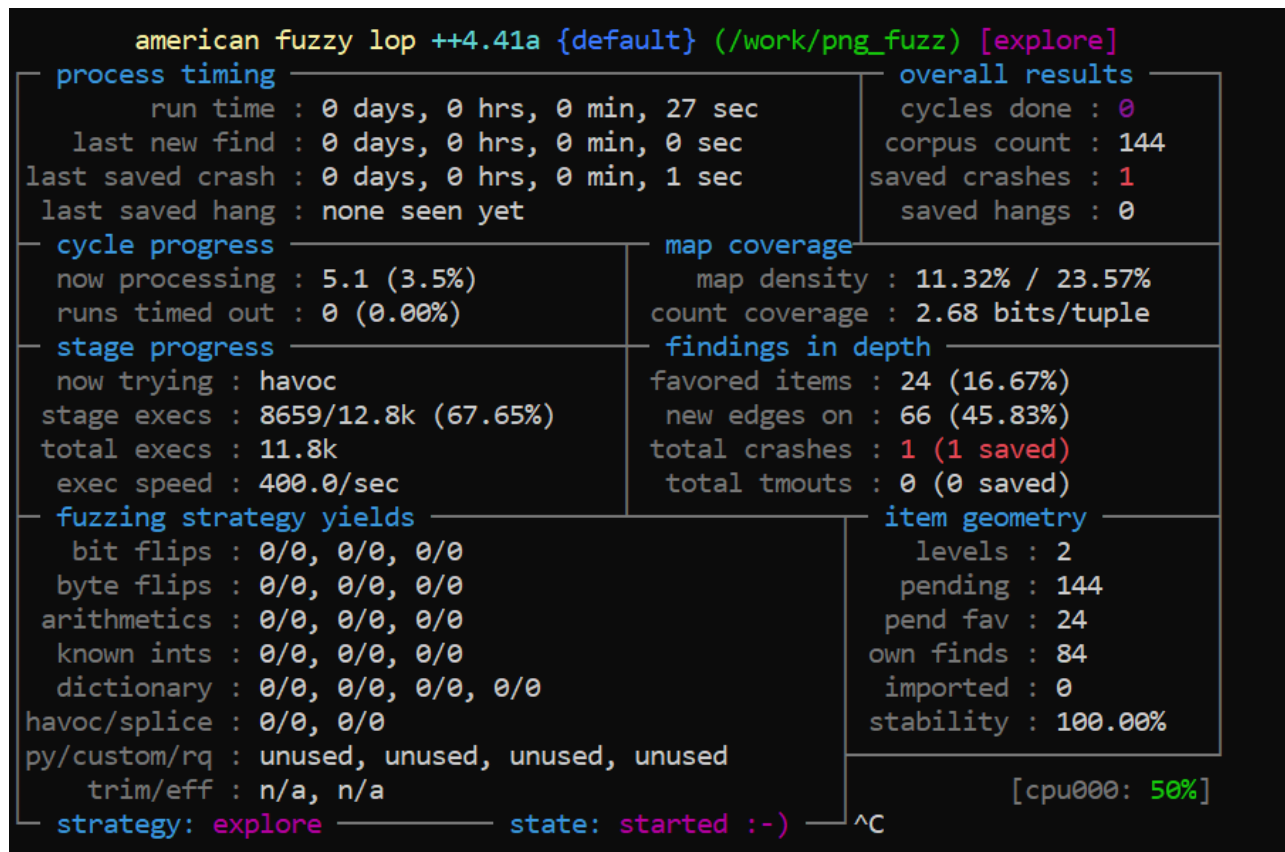


Figure 3: AFL++ status screen showing the synthetic off-by-one bug detected.

```

==1==ERROR: AddressSanitizer: heap-buffer-overflow on address 0x502000000014 at pc 0x5850a2fafc7a bp 0x7fff41c30650 sp 0x7fff41c30648
WRITE of size 1 at 0x502000000014 thread T0
#0 0x5850a2fafc79 in png_read_info /work/third_party/libpng-1.2.56/pngread.c:450:19
#1 0x5850a2fa4aee in main /work/src/harness.c:67:5
#2 0x7b9b8ac4f1c9 (/lib/x86_64-linux-gnu/libc.so.6+0x2a1c9) (BuildId: 8e9fd827446c24067541ac5390e6f527fb5947bb)
#3 0x7b9b8ac4f28a in __libc_start_main (/lib/x86_64-linux-gnu/libc.so.6+0x2a28a) (BuildId: 8e9fd827446c24067541ac5390e6f527fb5947bb)
#4 0x5850a2ecb4c4 in _start (/work/png_fuzz+0x314c4) (BuildId: 3e0e0fa368f89e3ee6958ecd2505e2e360167673)

0x502000000014 is located 0 bytes after 4-byte region [0x502000000010,0x502000000014)
allocated by thread T0 here:
#0 0x5850a2f66313 in malloc (/work/png_fuzz+0xccc313) (BuildId: 3e0e0fa368f89e3ee6958ecd2505e2e360167673)
#1 0x5850a2fdda58 in png_malloc_default /work/third_party/libpng-1.2.56/pngmem.c:511:13
#2 0x5850a2fdda58 in png_malloc /work/third_party/libpng-1.2.56/pngmem.c:468:14

SUMMARY: AddressSanitizer: heap-buffer-overflow /work/third_party/libpng-1.2.56/pngread.c:450:19 in png_read_info

```

Figure 4: ASan report confirming the synthetic off-by-one bug as a heap buffer overflow inside `png_read_info()`.

Metric	White-box	QEMU
Wall-clock time	2397 s	2398 s
Exec speed	535.1 exec/s	440.74 exec/s
Total executions	898k	1.06M
Edges discovered	888 / 3114	2375 / 65536
Corpus count	624	703
Map density	28.52%	3.62%
Cycles done	5	6
Stability	100.00%	100.00%
Target mode	default	qemu

Table 1: Comparison between the white-box and binary-only QEMU-mode campaigns. Values are taken from the AFL++ screenshots when visible; the QEMU average speed is taken from the internal logs because the status screen was temporarily idle and displayed 0.00 exec/s.

Target	Total edges	Reached	Peak density
libpng via pngtest_libpng_asan	4288	≈ 1149	26.8%
final harness png_fuzz	3114	≈ 660	21.2%

Table 2: Instrumentation-depth comparison between libpng’s own driver and the final fuzzing harness. Peak densities are taken from the AFL++ screenshots; total edge counts are taken from internal logs.

Configuration	Speed	Execs	Notes
No sanitizer + fork mode	1212.97 exec/s	363913	Fast fork-mode baseline; values from internal logs due to UI/capture issue
ASan + fork mode	529.1 exec/s	32.0k	Slower due to sanitizer overhead; value visible in the screenshot
ASan + persistent mode	1372 exec/s	82.2k	Fastest because persistent mode avoids most fork/exec reset cost

Table 3: Performance comparison of the final harness under different fuzzing configurations. Screenshot values are used when available; the no-sanitizer fork-mode values are taken from internal logs because the corresponding UI screenshot was incorrect.

```

Test case count : 28 favored, 0 variable, 0 ignored, 148 total
Bitmap range : 483 to 662 bits (average: 563.26 bits)
Exec timing : 2603 to 3846 us (average: 2813 us)

[*] No -t option specified, so I'll use an exec timeout of 20 ms.
[+] All set and ready to roll!

american fuzzy lop ++4.41a {default} (/work/pngtest_libpng_asan) [explore]
┌────────── process timing ───────────┐ ┌────────── overall results ───────────┐
│ run time : 0 days, 0 hrs, 0 min, 10 sec │ │ cycles done : 0 │
│ last new find : 0 days, 0 hrs, 0 min, 2 sec │ │ corpus count : 200 │
│ last saved crash : none seen yet │ │ saved crashes : 0 │
│ last saved hang : none seen yet │ │ saved hangs : 0 │
└────────── cycle progress ───────────┐ ┌────────── map coverage ───────────┐
│ now processing : 2.1 (1.0%) │ │ map density : 12.45% / 26.68% │
│ runs timed out : 0 (0.00%) │ │ count coverage : 3.04 bits/tuple │
└────────── stage progress ───────────┐ ┌────────── findings in depth ───────────┐
│ now trying : arith 32/8 │ │ favored items : 28 (14.00%) │
│ stage execs : 271/130k (0.21%) │ │ new edges on : 61 (30.50%) │
│ total execs : 5521 │ │ total crashes : 0 (0 saved) │
│ exec speed : 480.9/sec │ │ total tmouts : 0 (0 saved) │
└────────── fuzzing strategy yields ───────────┐ ┌────────── item geometry ───────────┐
│ bit flips : 1/7528, 1/7527, 0/7525 │ │ levels : 2 │
│ byte flips : 0/941, 0/940, 0/938 │ │ pending : 200 │
│ arithmetics : 5/65.7k, 0/130k, 0/0 │ │ pend fav : 28 │
│ known ints : 0/0, 0/0, 0/0 │ │ own finds : 52 │
│ dictionary : 0/0, 0/0, 0/0, 0/0 │ │ imported : 0 │
│ havoc/splice : 0/0, 0/0 │ │ stability : 100.00% │
│ py/custom/rq : unused, unused, unused, unused │ │ │
│ trim/eff : n/a, 98.41% │ │ │
└────────── strategy: explore ───────────┐ ┌────────── state: started :-) ───────────┐

```

Figure 7: AFL++ startup screen for pngtest_libpng_asan, reporting the total instrumented edge count for the libpng driver.

```

american fuzzy lop ++4.41a {default} (/work/png_fuzz) [explore]
  process timing
    run time : 0 days, 0 hrs, 0 min, 10 sec
    last new find : 0 days, 0 hrs, 0 min, 0 sec
    last saved crash : none seen yet
    last saved hang : none seen yet
  cycle progress
    now processing : 2.1 (1.0%)
    runs timed out : 0 (0.00%)
  stage progress
    now trying : havoc
    stage execs : 3461/12.8k (27.04%)
    total execs : 6193
    exec speed : 0.00/sec (zzzz...)
  fuzzing strategy yields
    bit flips : 0/0, 0/0, 0/0
    byte flips : 0/0, 0/0, 0/0
    arithmetics : 0/0, 0/0, 0/0
    known ints : 0/0, 0/0, 0/0
    dictionary : 0/0, 0/0, 0/0, 0/0
    havoc/splice : 0/0, 0/0
    py/custom/rq : unused, unused, unused, unused
    trim/eff : n/a, n/a
  strategy: explore
  overall results
    cycles done : 0
    corpus count : 200
    saved crashes : 0
    saved hangs : 0
  map coverage
    map density : 9.25% / 21.26%
    count coverage : 3.09 bits/tuple
  findings in depth
    favored items : 27 (13.50%)
    new edges on : 65 (32.50%)
    total crashes : 0 (0 saved)
    total tmouts : 0 (0 saved)
  item geometry
    levels : 2
    pending : 199
    pend fav : 27
    own finds : 52
    imported : 0
    stability : 100.00%
  [cpu014: 13%]
  state: started :-)
```

Figure 8: AFL++ startup/status screen for the final `png_fuzz` harness, reporting the total instrumented edge count for the fuzzing harness.

```

Test case count : 29 favored, 0 variable, 1 ignored, 148 total
  Bitmap range : 255 to 352 bits (average: 302.51 bits)
  Exec timing  : 1890 to 2297 us (average: 2002 us)

[*] No -t option specified, so I'll use an exec timeout of 20 ms.
[+] All set and ready to roll!

    american fuzzy lop ++4.41a {default} (/work/png_fuzz) [explore]
┌───────────┴───────────┐
┌ process timing ────────────┐ ┌ overall results ────────────┐
│   run time  : 0 days, 0 hrs, 1 min, 0 sec │ │ cycles done  : 0 │
│ last new find : 0 days, 0 hrs, 0 min, 0 sec │ │ corpus count : 254 │
│ last saved crash : none seen yet │ │ saved crashes : 0 │
│ last saved hang  : none seen yet │ │ saved hangs  : 0 │
└───────────┴───────────┘ └───────────┴───────────┘
┌───────────┴───────────┐ ┌ map coverage ────────────┐
│ cycle progress ────────────┐ │ map density  : 9.83% / 22.86% │
│ now processing  : 34.1 (13.4%) │ │ count coverage : 3.16 bits/tuple │
│ runs timed out  : 0 (0.00%) │ │ ────────────┴───────────┘
└───────────┴───────────┘ ┌ findings in depth ────────────┐
│ stage progress ────────────┐ │ favored items  : 76 (29.92%) │
│ now trying  : havoc │ │ new edges on  : 92 (36.22%) │
│ stage execs  : 10.3k/51.2k (20.18%) │ │ total crashes : 0 (0 saved) │
│ total execs  : 32.0k │ │ total tmouts  : 0 (0 saved) │
│ exec speed   : 529.1/sec │ │ ────────────┴───────────┘
└───────────┴───────────┘ ┌ item geometry ────────────┐
│ fuzzing strategy yields ────────────┐ │ levels  : 2 │
│ bit flips  : 2/5616, 2/5614, 1/5610 │ │ pending  : 248 │
│ byte flips : 0/702, 0/700, 0/696 │ │ pend fav : 74 │
│ arithmetics : 1/49.0k, 0/97.0k, 0/96.2k │ │ own finds : 106 │
│ known ints  : 0/6276, 0/26.5k, 0/38.9k │ │ imported  : 0 │
│ dictionary  : 0/6318, 0/6336, 0/0, 0/0 │ │ stability : 100.00% │
│ havoc/splice : 48/14.2k, 0/0 │ │ ────────────┴───────────┘
│ py/custom/rq : unused, unused, unused, unused │ │
│ trim/eff  : 24.46%/1236, 98.72% │ │
└───────────┴───────────┘ ┌───────────┴───────────┐
│ strategy: explore ────────────┐ │ state: started :-) │
└───────────┴───────────┘ └───────────┴───────────┘

```

Figure 9: AFL++ status screen for the ASan fork-mode performance measurement.

```
[*] No -t option specified, so I'll use an exec timeout of 20 ms.
[+] All set and ready to roll!
```

process timing		overall results	
run time : 0 days, 0 hrs, 1 min, 0 sec		cycles done : 0	
last new find : 0 days, 0 hrs, 0 min, 0 sec		corpus count : 280	
last saved crash : none seen yet		saved crashes : 0	
last saved hang : none seen yet		saved hangs : 0	
cycle progress	map coverage		
now processing : 78.1 (27.9%)	map density : 10.36% / 23.12%		
runs timed out : 0 (0.00%)	count coverage : 3.14 bits/tuple		
stage progress	findings in depth		
now trying : havoc	avored items : 84 (30.00%)		
stage execs : 1621/12.8k (12.66%)	new edges on : 104 (37.14%)		
total execs : 82.2k	total crashes : 0 (0 saved)		
exec speed : 1372/sec	total tmouts : 0 (0 saved)		
fuzzing strategy yields	item geometry		
bit flips : 0/7448, 0/7445, 0/7439	levels : 2		
byte flips : 0/931, 0/928, 0/922	pending : 267		
arithmetics : 0/65.1k, 0/129k, 0/128k	pend fav : 78		
known ints : 0/8360, 0/35.2k, 0/51.6k	own finds : 132		
dictionary : 0/8379, 0/8406, 0/0, 0/0	imported : 0		
havoc/splice : 65/66.4k, 0/0	stability : 99.72%		
py/custom/rq : unused, unused, unused, unused			
trim/eff : 30.95%/1832, 99.68%			
strategy: explore	state: started :-)	[cpu014: 20%]	

13

References

- [1] Firefox source code. <https://github.com/mozilla-firefox/firefox/blob/main/image/decoders/nsPNGDecoder.cpp>. [Online; accessed 14.05.2026].
- [2] Firefox source code. <https://github.com/mozilla-firefox/firefox/blob/main/media/libpng/pngpread.c>. [Online; accessed 14.05.2026].
- [3] Imagemagick. <https://imagemagick.org/#gsc.tab=0>. [Online; accessed 13.05.2026].
- [4] Imagemagick source code. <https://github.com/ImageMagick/ImageMagick/blob/main/coders/png.c>. [Online; accessed 14.05.2026].
- [5] VAN SCHAİK, W. Pngsuite. <https://www.libpng.org/pub/png/pngsuite-all-good.html>, 2011. [Online; accessed 05.05.2026].